1. High-level design

> One agent per viewer. On every app open it decides one measurable goal and one proportionate action (or nothing), proves every step with evidence, stays inside a strict time budget, and afterwards measures honestly whether it helped.

1.1 The problem in one paragraph

An OTT app for regional-dialect content wants an "agent" that owns each viewer's next step. When a viewer opens the app, the agent has seconds to decide whether to change their home page, send one message later, or do nothing. Later it must judge from real signals whether that worked, and prove the effect against a randomized holdout.

The data is deliberately messy:

- events arrive late, duplicated, or with wrong device clocks;
- profiles are a day stale or duplicated;
- three sources disagree about the viewer's dialect;
- two goal rules are vague and one success signal never appears in the data;
- the replay injects surprises: outages, a corrected rule, a model that lies, catalogue text that tries to instruct the agent.

1.2 What "good" means here (from the brief's scoring)

Weight	Area	What we optimised for
20%	Goal and decision quality	One eligible, measurable goal per viewer; proportionate actions; "do nothing" when that is right
15%	Evidence and state	Every fact has a source and an age; conflicts stay visible; durable per-viewer state
15%	Real-time reliability	Within 60 s (in practice ~1 s); duplicates handled idempotently; safe fallbacks
15%	Verification and safety	An independent gate that can say no; holdout, consent, quiet hours, caps, dialect, injection
15%	Measurement and learning	Attributable outcomes; intent-to-treat (ITT) vs holdout; lessons fed back
10%	Engineering quality	One command to run; 75 tests; reproducible
10%	Use of AI tools	AI-built and human-checked; mutation testing; rejected suggestions logged

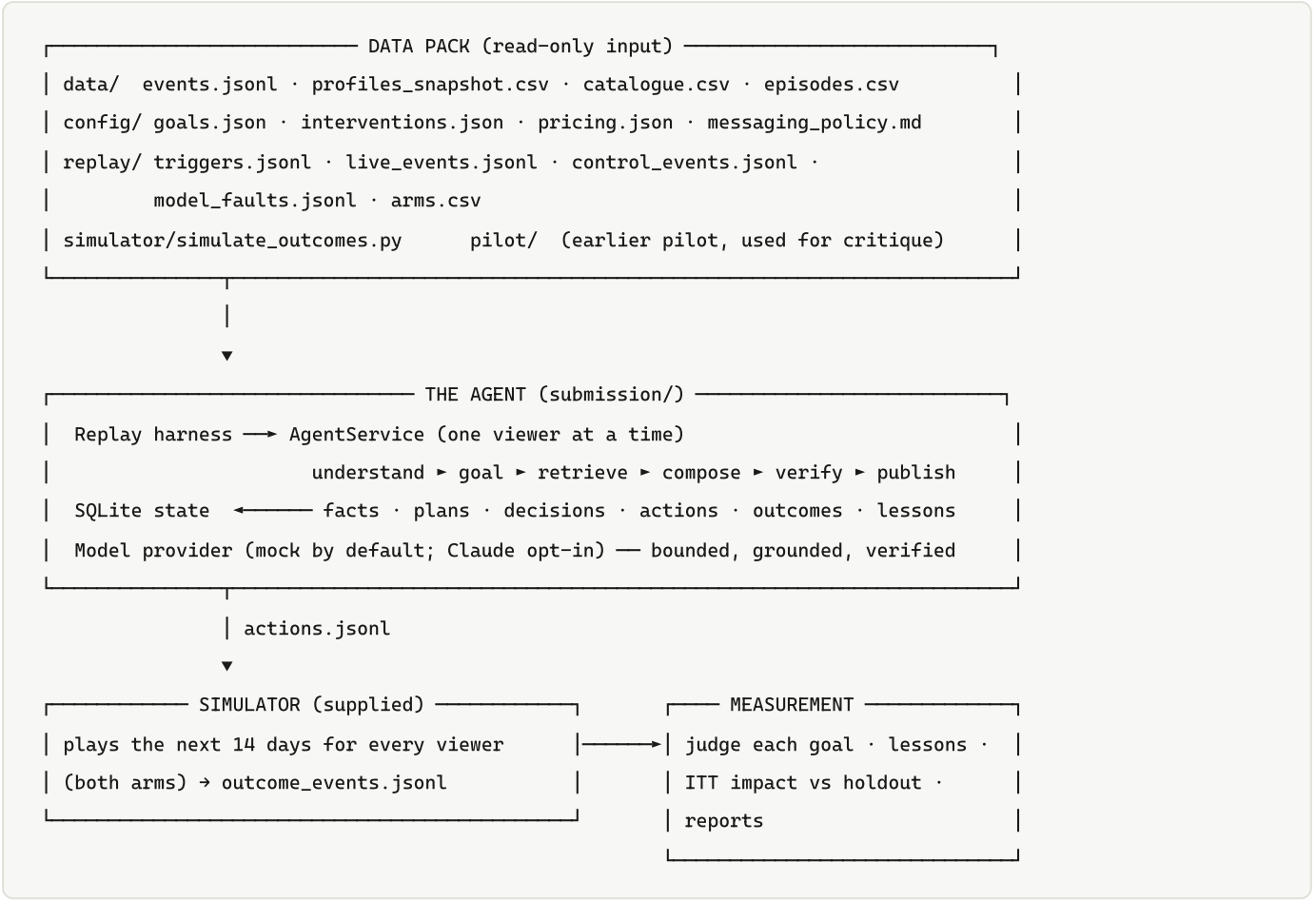
1.3 Design principles (the "why" behind everything)

1. **Code decides what is allowed; the model judges within it.** Eligibility, consent, dialect and availability are computed by code from data and versioned config. The model only chooses among options the code

produced.

2. **Evidence or nothing.** Every claim in a decision points to an event, a profile row, a catalogue row, a rule version or a policy version, with its age. Model text is never evidence.
3. **Two independent minds.** A planner proposes; a separate verifier, written differently and reading raw data, approves. It can and does disagree.
4. **Fast by design, not by luck.** A page only helps if it arrives before the viewer reaches home (seconds), so the page path is deterministic and model calls are deadline-aware.
5. **Fail closed.** No arm, no facts, no budget, no consent: publish nothing, and record why.
6. **Measure honestly.** Compare randomized arms (ITT), never "served vs unserved". Report a null result as a null result.
7. **Behaviour lives in config, not code.** Goal readings, thresholds, priorities, copy and safety patterns are in a versioned policy file that every decision cites.

1.4 System context



1.5 Major components

Component	Responsibility	Key idea
Replay harness	Feeds triggers, live events and control events in delivery order; models time and queues	Virtual clock: `published_at = delivered + queue wait + dependency time + CPU`

Component	Responsibility	Key idea
AgentService	The per-viewer agent: runs the decision pipeline and owns plans	One decision at a time per viewer; one open plan per viewer
Reconciler	Turns raw rows into facts with source, time and freshness; resolves conflicts	Precedence per fact, never "newest wins"
Goal engine	Generic eligibility checks and success matchers, from `goals.json` plus the policy	No goal ids in code; vague rules refused unless reviewed
Retriever (RAG)	BM25 over the catalogue, queried with the viewer's own searches and plays	Only titles the viewer can be shown; cited as evidence
Model advisor	Three bounded tasks: choose a goal, compose the page, write the copy	Structured JSON; grounded to offered options; fallback to rules
Planner	Builds candidate rows, the model payload, the grounding check and follow-ups	Proportionate: page now, at most one message later
Verifier	Independent gate: 20+ checks re-derived from raw data	The only path to publication
Dependency layer	Budgets, timeouts, circuit breaker, fault injection, load shedding	Failures degrade to rules or nothing, on time
Store	SQLite state; invariants enforced by the schema	One open plan per viewer, one message per day: database-enforced
Measurement	Judge, lessons, ITT impact, reports, audit	Attribution corrects for late uploads

1.6 The life of one decision

```
App open (trigger)
  |
  ├─ 0 Gate ──────────── seen this event_id before? → return the stored decision (idempotent)
  |                      no experiment arm? → FAILED_SAFE (fail closed)
  ├─ 1 Understand ─────── facts + source + age; conflicts kept visible
  |                      buffered > 10 min on the phone? → NO_ACTION (history, not a visit)
  ├─ 2 Plan upkeep ────── keep / close / revise the single open plan
  ├─ 3 Goal ──────────── 12 goals checked by rules → model may choose among the eligible ones
  |                      none eligible → NO_ACTION with every goal's reason
  ├─ 4 Holdout? ──────── shadow goal recorded, NO_ACTION (keeps measurement fair)
  ├─ 5 Retrieve ──────── candidates from the viewer's searches + plays, popularity, recency
  ├─ 6 Compose ───────── model picks hero + rows within the candidates → grounding check
  |                      (+ one follow-up message if the policy allows; model may decline)
  ├─ 7 Verify ────────── independent checks → repair / drop / block
  └─ 8 Publish ───────── one transaction: decision + facts + plan + actions
```

Later: simulator, then judge (succeeded / failed / inconclusive), then lessons for the viewer, then ITT impact.

1.7 Where the "AI" is, and where it deliberately is not

Uses a model	Never uses a model
Choosing among goals already proven eligible	Deciding eligibility, consent, dialect, availability
Composing the home page from retrieved candidates	Inventing titles, prices, offers, deadlines
Writing push copy (then verified)	Approving its own output
	Reading catalogue text as instructions

Retrieval-augmented generation (RAG): the viewer's own signals retrieve candidates, the candidates are given to the model as data, and the model's answer is checked against them. With the default mock provider the pipeline runs end to end without a key. Claude runs it for real with `--llm anthropic`.

1.8 How each surprise in the brief is handled

Surprise	Response
Burst of buffered offline opens	`sent_at − occurred_at` > 10 min means a past visit: recorded as history, no action (424 in the sample)
Never-paid profile, then a purchase 2 minutes later	Live event closes the open plan at once (succeeded if it happened after the goal started)
App setting, profile and campaign link name three dialects	App setting wins (the app shows that dialect); the others are kept as conflicts
Goal success condition corrected mid-replay	New rule version; only open plans of that goal are updated; closed outcomes untouched; survives restart
Model claims eligibility without evidence	Rejected: not in the eligible set; logged with the rule's reason
Catalogue text tells the agent to message everyone	Title quarantined from promotion; its text never reaches the model; no action can come from data
Model or data store slow/unavailable	Circuit breaker, timeouts, cached snapshot (page only), or a safe no-action
Traffic triples at peak	Worker-pool queue; deadline-aware model calls; load shedding when the queue backs up

1.9 Results of the sample run (generated, not typed)

Metric	Value
Triggers / decisions / duplicate deliveries	2,769 / 2,687 / 90 answered idempotently
Published actions	920 home pages, 223 push, 11 WhatsApp
Simulator policy problems	**0**
Latency	p50 0.44 s · p95 1.25 s · max 6.9 s · **0** over 60 s · 910/912 pages within 3 s
Model tasks	897 page compositions, 411 goal choices, 217 copies; every failure fell back safely

Metric	Value
Retrieval	707 of 920 pages contain titles retrieved from the viewer's own signals
Impact (ITT, first goal)	Agent 13.3% vs holdout 12.3%, **+1.0 pt, 95% CI −2.4 to +4.4**: no detectable effect, reported as such
Pilot report's "2.3× uplift"	+22.4 pts served-vs-everyone-else; the same file's randomized arms give **−1.3 pts**
Evaluation check (`python -m oua check --run-tests`)	**48 of 48** checks against the brief pass (see `sample_run/evaluation_check.md`)

1.10 Key trade-offs (be ready to defend these)

Decision	Alternative rejected	Why
Rules decide eligibility; the model chooses within	Model decides everything	The brief penalises asserted eligibility; model text is not evidence
BM25 retrieval	Vector DB / embeddings	Queries are title words, lexical search fits; offline, no key, nothing leaves the machine
SQLite, single process	Kafka / Redis / microservices	Assignment scale; invariants enforced by the schema; concurrency modelled in virtual time
Shadow goals for the holdout	Measure only served viewers	Same goal logic in both arms makes ITT fair
Refuse vague goals without a reviewed reading	Let code or a model guess	Guessing creates unmeasurable or wrong goals
Mock provider by default	Require an API key	Evaluators must run it; privacy of the pack

1.11 Answers to the interview questions in the brief

"How does the agent decide what it knows is current enough?"

- Every fact carries its source and age.
- Server time orders events; device time is used only to detect buffering.
- Billing events received after the profile export override the profile.
- If the data store is down, a cached snapshot is used, but only for a page, never for messages (consent and caps can't be re-checked).

"When should it do nothing, and how do you know that was right?"

- It does nothing for: holdout, a stale open, no eligible goal, a dialect with no content, a page already live, verifier rejection, or missing facts.
- Each case is a recorded decision with a reason.
- Whether doing nothing was right is checked by comparing to the holdout, which by construction does nothing.

"How do you show the agent caused the change?"

- Intent-to-treat on randomized arms, with shadow goals so both arms are judged identically.

- A pre-treatment balance table shows the arms looked alike before the agent acted.
- Served-vs-unserved is shown only to demonstrate its selection bias (+14 pts that are not real).

"1,000 opens in the same minute?"

- Worker pool plus per-viewer locks.
- Queue wait is recorded on every decision.
- Optional model calls are shed when the queue passes 5 s; pages stay deterministic and fast.
- Tested with a 300-open burst.

"Which decisions should never be automated?"

- Prices and offers, new message templates, un-quarantining flagged content, and goal-rule changes.
- The agent only applies versions that humans supply.

"10 million viewers, 40 dialects?"

- Partition by viewer ID (a keyed stream).
- A key-value store for state with conditional writes instead of SQLite constraints.
- Per-dialect precomputed candidates and search indexes.
- Model calls only where an A/B test shows they help.

"What did the AI assistant suggest that you didn't accept?"

- A vector database.
- Letting the model decide eligibility or pick titles freely.
- OpenAI-compatible shims for Claude.
- Running a live model replay automatically.
- The "new episode" WhatsApp template, because its claim can't be verified.